

RAG and Embeddings Basics

1. What is RAG?

Retrieval-Augmented Generation, or RAG, combines information retrieval with large language models. Instead of relying only on information learned during model training, an application retrieves relevant information from an external knowledge source.

A basic RAG pipeline contains document loading, text splitting, embedding generation, vector storage, retrieval, prompt construction, and response generation.

2. Document Loading

Documents can come from PDFs, text files, web pages, databases, or other sources. A document loader converts source data into a representation that an application can process.

Metadata such as the source filename, page number, author, or document date can be stored alongside document content. This metadata can later support filtering and source attribution.

3. Text Chunking

Large documents are usually divided into smaller chunks before creating embeddings. Chunk size affects both retrieval quality and the amount of context provided to the language model.

Chunk overlap can preserve information that occurs near the boundary between two chunks.

4. Embeddings and Vector Search

An embedding is a numerical representation of text that captures semantic information. Similar pieces of text tend to have embeddings that are close together in vector space.

A vector database can store embeddings and retrieve chunks that are semantically similar to a user's query.

5. Retrieval and Generation

When a user asks a question, the query is converted into an embedding and relevant document chunks are retrieved. These chunks are then supplied as context to a language model.

The quality of a RAG system depends on several components, including document quality, chunking strategy, embedding model, retrieval method, context selection, and the final generation prompt.

6. Simple RAG Example

Suppose a company stores its employee handbook as a PDF. A user asks, 'How many days of annual leave are available?' The RAG system retrieves the relevant section of the handbook and gives that text to the language model so it can formulate an answer based on the retrieved information.

This approach can reduce the need to retrain a language model whenever the external knowledge base changes.